

```
In [110... import pandas
inventory = pandas.read_csv ('wk8-inventory.csv')
sales = pandas.read_csv ('wk8-sales.csv')
stores = pandas.read_csv ('wk8-stores.csv')
products = pandas.read_csv ('wk8-products.csv')
customers = pandas.read_csv ('wk8-customers.csv')
dataframe
products.head(3)
```

```
Out[110... 
```

	product_id	product_name	category	subcategory	brand	price	cost	weight
0	1	Apple iPhone 13	Electronics	Smartphones	Apple	899.99	649.99	0.45
1	2	Samsung Galaxy S21	electronics	Smartphones	Samsung	799.99	539.99	0.50
2	3	Sony WH-1000XM4	ELECTRONICS	Headphones	Sony	349.99	210.00	0.60

```
In [112... inventory.head(3)
```

```
Out[112... 
```

	inventory_id	store_id	product_id	quantity_in_stock	last_restock_date	reorder_level
0	1	1	1	25	2022-10-15	10.0
1	2	1	5	42	2022-11-02	15.0
2	3	1	10	18	2022-09-30	8.0

```
In [138... inventory.describe()
```

```
Out[138... 
```

	inventory_id	store_id	product_id	quantity_in_stock	reorder_level
count	75.000000	75.000000	75.000000	75.000000	69.000000
mean	38.000000	8.000000	12.733333	22.640000	12.681159
std	21.794495	4.349588	7.179801	13.923187	3.327659
min	1.000000	1.000000	1.000000	0.000000	8.000000
25%	19.500000	4.000000	6.500000	15.000000	10.000000
50%	38.000000	8.000000	13.000000	25.000000	10.000000
75%	56.500000	12.000000	19.000000	33.500000	15.000000
max	75.000000	15.000000	25.000000	45.000000	20.000000

```
In [140... inventory.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 75 entries, 0 to 74
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   inventory_id          75 non-null    int64
1   store_id              75 non-null    int64
2   product_id            75 non-null    int64
3   quantity_in_stock     75 non-null    int64
4   last_restock_date     66 non-null    object
5   reorder_level         69 non-null    float64
dtypes: float64(1), int64(4), object(1)
memory usage: 3.6+ KB
```

In [114...

sales.head(3)

Out[114...

	sale_id	date	store_id	customer_id	product_id	quantity	total	payment_methc
0	1	2022-01-15	3.0	12	5	2	49.98	Credit Cai
1	2	2022-01-16	1.0	5	10	1	49.99	credit cai
2	3	2022-01-18	2.0	8	3	1	349.99	DEBIT CAR

In [144...

sales.describe()

Out[144...

	sale_id	store_id	customer_id	product_id	quantity	total
count	40.000000	39.000000	40.000000	40.000000	40.000000	40.000000
mean	20.500000	6.487179	14.075000	13.500000	1.625000	352.296500
std	11.690452	4.235637	8.303189	8.857852	0.952392	377.625776
min	1.000000	1.000000	1.000000	1.000000	1.000000	19.980000
25%	10.750000	3.000000	6.750000	5.750000	1.000000	99.737500
50%	20.500000	6.000000	13.500000	11.500000	1.000000	229.985000
75%	30.250000	9.500000	20.250000	21.250000	2.000000	352.487500
max	40.000000	15.000000	29.000000	30.000000	5.000000	1599.990000

In [142...

sales.describe()

Out[142...

	sale_id	store_id	customer_id	product_id	quantity	total
count	40.000000	39.000000	40.000000	40.000000	40.000000	40.000000
mean	20.500000	6.487179	14.075000	13.500000	1.625000	352.296500
std	11.690452	4.235637	8.303189	8.857852	0.952392	377.625776
min	1.000000	1.000000	1.000000	1.000000	1.000000	19.980000
25%	10.750000	3.000000	6.750000	5.750000	1.000000	99.737500
50%	20.500000	6.000000	13.500000	11.500000	1.000000	229.985000
75%	30.250000	9.500000	20.250000	21.250000	2.000000	352.487500
max	40.000000	15.000000	29.000000	30.000000	5.000000	1599.990000

In [146...

```
sales.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40 entries, 0 to 39
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sale_id               40 non-null     int64
1   date                  39 non-null     object
2   store_id              39 non-null     float64
3   customer_id           40 non-null     int64
4   product_id            40 non-null     int64
5   quantity              40 non-null     int64
6   total                 40 non-null     float64
7   payment_method        39 non-null     object
dtypes: float64(2), int64(4), object(2)
memory usage: 2.6+ KB
```

In [124...

```
stores.head(3)
```

Out[124...

	store_id	store_name	address	city	state	zip_code	region	size_sqft	opening
0	1	City Center	123 Retail Blvd	New York	NY	10001	Northeast	32500.0	2005-
1	2	Mall of Chicago	456 Shopping Ave	Chicago	IL	60611	Midwest	45000.0	2007-
2	3	Los Angeles Plaza	789 Commerce St	Los Angeles	CA	90001	West	55000.0	2004-

In [148...

```
stores.describe()
```

Out[148...

	store_id	size_sqft
count	15.000000	15.000000
mean	8.000000	30653.846154
std	4.472136	10500.654087
min	1.000000	16500.000000
25%	4.500000	23000.000000
50%	8.000000	30653.846154
75%	11.500000	33750.000000
max	15.000000	55000.000000

In [150...

```
stores.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   store_id        15 non-null    int64
1   store_name      15 non-null    object
2   address         15 non-null    object
3   city            15 non-null    object
4   state           15 non-null    object
5   zip_code        15 non-null    object
6   region          15 non-null    object
7   size_sqft       15 non-null    float64
8   opening_date    14 non-null    object
dtypes: float64(1), int64(1), object(7)
memory usage: 1.2+ KB
```

In [118...

```
customers.head(3)
```

Out[118...

	customer_id	first_name	last_name	email	phone	address
0	1	James	Smith	james.smith@gmail.com	555-123-4567	123 Main St
1	2	Mary	Johnson	NaN	212.555.6789	456 Park Ave
2	3	John	Williams	jwilliams@yahoo.com	(555) 987-6543	789 Broadway Ave

In [152...

```
customers.describe()
```

Out[152...

	customer_id
count	31.000000
mean	15.838710
std	8.858505
min	1.000000
25%	8.500000
50%	16.000000
75%	23.500000
max	30.000000

In [154...

customers.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 31 entries, 0 to 30
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customer_id           31 non-null    int64
1   first_name            31 non-null    object
2   last_name             31 non-null    object
3   email                 28 non-null    object
4   phone                 29 non-null    object
5   address               31 non-null    object
6   city                  31 non-null    object
7   state                 31 non-null    object
8   zip_code              30 non-null    object
9   registration_date     28 non-null    object
dtypes: int64(1), object(9)
memory usage: 2.6+ KB
```

In [128...

```
largest_store_size = stores['size_sqft'].max()
print(largest_store_size)
```

55000.0

In [134...

```
#2.1 - replace missing values with means for numeric values, and most common (mode) for
#Ex 1: Numeric values- Replace missing size_sqft with average size_sqft
```

```
stores['size_sqft'] = stores['size_sqft'].fillna(stores['size_sqft'].mean())
stores.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   store_id        15 non-null    int64
1   store_name      15 non-null    object
2   address         15 non-null    object
3   city            15 non-null    object
4   state           15 non-null    object
5   zip_code        15 non-null    object
6   region          15 non-null    object
7   size_sqft       15 non-null    float64
8   opening_date    14 non-null    object
dtypes: float64(1), int64(1), object(7)
memory usage: 1.2+ KB
```

```
In [160... #3.1 unique products
products['product_id'].nunique()
```

```
Out[160... 30
```

```
In [182... #3.2 five most expensive products
top_5_most_expensive = products.sort_values(by="price", ascending=False).head(5)
top_5_most_expensive
```

```
Out[182...
   product_id  product_name  category  subcategory  brand  price  cost  weight
3           4    Dell XPS 13  Electronics  Laptops    Dell  1299.99  899.99   2.80
0           1  Apple iPhone 13  Electronics  Smartphones  Apple   899.99  649.99   0.45
1           2    Samsung Galaxy S21  electronics  Smartphones  Samsung   799.99  539.99   0.50
19          20    Dyson V11 Vacuum  HOME & KITCHEN  Appliances    Dyson   599.99  375.00   6.70
26          27    Sony PlayStation 5  Electronics  Gaming      Sony   499.99  399.00   4.50
```

```
In [196... #3.3 store with largest floor space
largest_stores_sqft = stores.sort_values(by="size_sqft", ascending=False).head(1)
largest_stores_sqft
```

```
Out[196...
   store_id  store_name  address  city  state  zip_code  region  size_sqft  opening_da
2           3  Los Angeles Plaza  789 Commerce St  Los Angeles  CA  90001  West  55000.0  2004-03-
```

```
In [206... #3.4 Get counts_of_customers_by_states  
state_distribution = customers['state'].value_counts()  
state_distribution
```

```
Out[206... state  
TX          4  
CA          3  
NY          2  
MA          1  
NM          1  
WI          1  
KY          1  
MD          1  
Maryland    1  
TN          1  
MI          1  
NV          1  
Oregon      1  
Washington  1  
CO          1  
California  1  
IN          1  
North Carolina 1  
OH          1  
FL          1  
Texas       1  
PA          1  
AZ          1  
IL          1  
MO          1  
Name: count, dtype: int64
```

```
In [226... #Ex1 Numeric value - replace missing size_sqft with ave size_sqft  
  
stores['size_sqft'] = stores['size_sqft'].fillna(stores['size_sqft'].mean())  
stores.info()
```

KeyError

Traceback (most recent call last)

Cell In[226], line 3

```

1 #Ex1 Numeric value - replace missing size_sqft with ave size_sqft
----> 3 stores ['size_sqft'] = stores['size_sqft'].fillna(stores['size_sqft'].mean
())
4 stores.info()

```

File ~\anaconda3\Lib\site-packages\pandas\core\series.py:1121, in Series.__getitem__(self, key)

```

1118     return self._values[key]
1120 elif key_is_scalar:
-> 1121     return self._get_value(key)
1123 # Convert generator to list before going through hashable part
1124 # (We will iterate through the generator there to check for slices)
1125 if is_iterator(key):

```

File ~\anaconda3\Lib\site-packages\pandas\core\series.py:1237, in Series._get_value(self, label, takeable)

```

1234     return self._values[label]
1236 # Similar to Index.get_value, but we do not fall back to positional
-> 1237 loc = self.index.get_loc(label)
1239 if is_integer(loc):
1240     return self._values[loc]

```

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\range.py:417, in RangeIndex.get_loc(self, key)

```

415     raise KeyError(key) from err
416 if isinstance(key, Hashable):
--> 417     raise KeyError(key)
418 self._check_indexing_error(key)
419 raise KeyError(key)

```

KeyError: 'size_sqft'

In [232...

```

sales ['payment_method'] = sales['payment_method'].fillna()
sales.info()

```



```

-----
ValueError                                Traceback (most recent call last)
Cell In[232], line 1
----> 1 sales['payment_method'] = sales['payment_method'].fillna()
      2 sales.info()

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:7293, in NDFrame.fillna(self, value, method, axis, inplace, limit, downcast)
    7286         if ctr <= ref_count:
    7287             warnings.warn(
    7288                 _chained_assignment_warning_method_msg,
    7289                 FutureWarning,
    7290                 stacklevel=2,
    7291             )
-> 7293 value, method = validate_fillna_kwargs(value, method)
    7294 if method is not None:
    7295     warnings.warn(
    7296         f"{type(self).__name__}.fillna with 'method' is deprecated and "
    7297         "will raise in a future version. Use obj.ffmpeg() or obj.bfill() "
    7298         (...)
    7300         stacklevel=find_stack_level(),
    7301     )

File ~\anaconda3\Lib\site-packages\pandas\util\_validators.py:293, in validate_fillna_kwargs(value, method, validate_scalar_dict_value)
    290 from pandas.core.missing import clean_fill_method
    292 if value is None and method is None:
--> 293     raise ValueError("Must specify a fill 'value' or 'method'.")
    294 if value is None and method is not None:
    295     method = clean_fill_method(method)

ValueError: Must specify a fill 'value' or 'method'.

```

In []: